

An End-to-End Benchmarking Tool for Analyzing the Hardware-Software Implications of Multi-modal DNNs

Tianhao Huang, Xiaozhi Zhu, Mo Niu
Shanghai Jiao Tong University

Abstract—Multi-modal deep neural networks (DNNs) have become increasingly pervasive in many machine learning application domains due to their superior accuracy by fusing various modalities together. However, multi-modal DNNs present many unique characteristics such as multi-stage execution, frequent synchronization and high heterogeneity, which are not well understood in the system and architecture community. In this article, we first present and characterize a set of multi-modal DNN workloads of different sizes from five domains and measure metrics like accuracy to ensure the availability of these applications from the algorithm perspective. We then explore their important hardware-software implications from system and architecture aspects by conducting an in-depth analysis on the unique hardware-software characteristics of multi-modal DNNs. We hope that our work can help guide future hardware-software design and optimization for efficient inference of multi-modal DNN applications on both cloud and edge computing platforms.

1. PROBLEM AND MOTIVATION

Multi-modal deep neural networks (DNNs) have attracted tremendous attention in recent years for its ability to outperform traditional uni-modal DNNs. Through fusing information from a variety of modalities, multi-modal DNNs achieve 5% to 30% improvements compared with SOTA (state of the art) uni-modal DNNs in a wide spectrum of scenarios [1]. Thus multi-modal DNNs are being deployed on multiple platforms from servers to edge devices to handle various important tasks such as multimedia, robotics and automatic driving.

However, despite their superiority, multi-modal DNN applications pose severe challenges to previously applied design and deployment methodologies. For example, multi-modal DNN networks present a clear three-stage execution mode (encoder-fusion-head), which causes resource under-utilization in certain stages. To understand these newly emerged issues, we build several end-to-end multi-modal DNN applications and conduct an in-depth analysis of their implications at the system and architecture level.

Previous research has given extensive attention to characterize features of different DNN applications, which can be classified into three types: algorithm-oriented DNN bench-

marks, system-oriented DNN benchmarks and architecture-oriented DNN benchmarks according to their associated evaluation metrics. However, the majority of the existing works only focus on uni-modal DNNs. For those who take multi-modal DNNs into consideration, they only provide algorithms-oriented analysis such as performance and complexity. There lacks characterization for multi-modal DNNs from architecture and system level.

In this paper, we first present and characterize a wide spectrum of multi-modal DNN workloads in terms of their end-to-end inference process. We implement them all in PyTorch leveraging existing model designs. We then set our system-level metrics and investigate the system and architecture implications of multi-modal DNN applications.

2. BACKGROUND AND RELATED WORK

Multi-modal DNN is a kind of neural network that learns and improves through the use and experience of data from multiple modalities. It can be applied to a wide range of applications in different fields.

Figure 1 briefly shows what a multi-modal DNN is. First, the encoder stage extracts features from data of different modalities respectively using various representation learning methods. After the encoding phase, the fusion stage generates a multi-modal representation by integrating the processed uni-modal data from different modalities. Finally, the multi-modal representation is passed through a task-specific network to generate the final prediction, which is referred to as the head stage.

Multiple benchmarks have been proposed to characterize DNN applications. Besides, in both academia and industry, researchers have proposed several simulation frameworks such as *GPGPU-sim* [2], *timeloop* [3] to simulate and analyze the features of different DNNs. However, most of them only focus on uni-modal DNNs. *MLPerf* [4] is a comprehensive benchmark for measuring machine learning (ML)

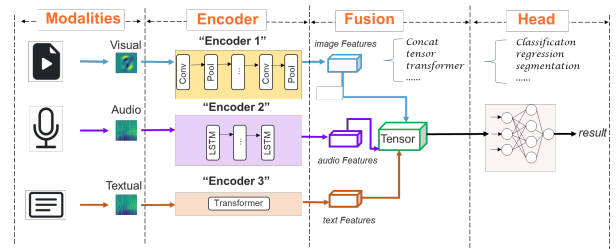


Figure 1: Structure of a multi-modal DNN.

Copyright is held by author/owner(s).

Table 1: Overview of different applications

Application domain	Multimedia Application		Affective Computing		Intelligent Medicine		Smart Robotics		Automatic Driving
Application	<i>AV-mnist</i>	<i>MM-imdb</i>	<i>CMU-mosei</i>	<i>MUSTARD</i>	<i>Medical VQA</i>	<i>Medical Seg.</i>	<i>Mujoco Push</i>	<i>Vision & Touch</i>	<i>TransFuser</i>
Modalities	1.image, 2.audio	1.image, 2.text	1.language, 2.vision, 3.audio	1.language, 2.vision, 3.audio	1.image, 2.text	MRI scans (T1, T1c, T2, Flair)	1.position, 2.sensor, 3.image, 4.control	1.image, 2.force, 3.proprioception, 4.depth	1.image 2.LiDAR
Encoders	1,2: LeNet	1: VGG 2: Albert	1: BERT 2: OpenFace 3: Librosa	1: BERT 2: OpenFace 3: Librosa	1: DenseNet 2: Roberta	All: U-Net	1,2,4: MLP 3: CNN	1,2,4: CNN 3: MLP	1,2: ResNet
Fusion methods	Concat, tensor	Concat, tensor	Concat, tensor, transformer	Concat, tensor, transformer	Transformer	Transformer	Concat, tensor, transformer	Concat, tensor	transformer

inference performance across a spectrum of use cases, while *TBD* [5] mainly focuses on machine learning training. A few architecture-oriented DNN benchmarks [6] analyzed the architectural implications of uni-modal DNNs but fail to analyze multi-modal DNNs. Although MultiBench analyzes multiple multi-modal DNNs from complexity, performance and robustness, it does not characterize their features from the perspective of system and architecture. Besides, it takes processed data as input in many scenarios, failing to provide a complete end-to-end multi-modal framework.

3. PROPOSED METHOD

3.1 Applications Overview

We evaluate on mainly nine multi-modal datasets from five domains, which cover traditional applications like multimedia and emerging domains such as autonomous driving, to investigate the characteristics of multi-modal DNNs. The detail of these applications are presented in Table 1.

3.2 Profiling Pipeline

We provides a set of profiling tools based on the most commonly used hardware available today (CPU and NVIDIA GPU) to effectively measure the evaluation metrics of multi-modal DNNs from the algorithm, system and architecture level performance. The overall profiling architecture is shown in Figure 2. To ensure the authority of the measured results, we measure the performance of the network based on standard tools such as *Pytorch Profiler*, *NVIDIA Nsight Compute* and *NVIDIA Nsight System*.

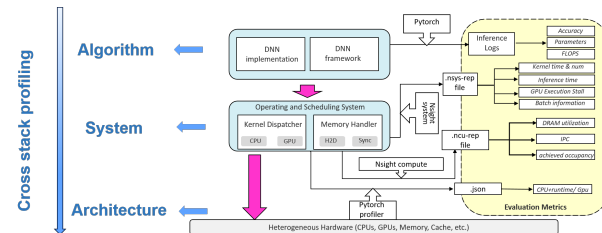


Figure 2: MMBench Toolchain / Evaluation Metrics

From algorithm perspective, we first reproduce the application we choose to ensure that we can achieve basic accuracy. After that we run python codes by calling the corresponding functions to get software metrics like the amount of parameters and flops.

And from the system and architecture perspective, We first figure out the GPU resource manner of multi-modal DNNs since GPU is the central component in learning tasks using 5 commonly used metrics: (1) DRAM utilization, (2) achieved occupancy, (3) IPC, (4) GLD efficiency and (5) GST efficiency. We then investigate how CPU works corre-

spondingly. Since CPUs seldom become the bottleneck in learning tasks, we mainly focus on the data operations and synchronization operations. Finally, we dive further into GPU kernels. We divide all operations into eight main categories and counts the number of the kernels launched for these operations.

4. PERFORMANCE EVALUATION

In this section, we present a detailed evaluation of our in-depth analysis of multi-modal DNNs from three main aspects: multi-stage execution, workload heterogeneity and execution synchronization.

4.1 stage-level analysis

We take five attributes representing GPU resource manner as mentioned above into consideration. In Figure 3, we find that in some cases, the encoder stage present higher DRAM utilization, IPC and GPU occupancy compared with fusion and head stages which usually take extracted features as input, while in other cases the fusion is so complex that it consumes more resources. In general, there exists significant time and resource imbalance in different stages, leading to possible resource under-utilization. If we assign resource according to the most resource-consuming stage, the majority of resources will stay idle in other stages.

4.2 heterogeneity analysis

We analyze the GPU kernel time breakdown of multi-modal DNNs in Figure 4. The GPU operation is classified into 8 categories, including convolutions (Conv), batch normalization (BNorm), element-wise operation (Elewise), pooling (Pooling), relu activation (Relu), general matrix multiply (Gemm), reduce (Reduce) and else (Other). We observe that the dominated operations vary significantly depending on the application, stage and encoder network. In the encoder stage of *AV-mnist*, convolutions account for 25.9%, while in *MM-imdb*, they comprise only 3%. Different stages have different networks, so it is natural that GPU kernel breakdowns also differ. For different encoder networks within the same application, such as *VGG* and *Albert* in *MM-imdb*, the kernel breakdown will also vary. In the case of *VGG*, Gemm dominates the breakdown at 72%, while for *Albert*, relu dominates at 66%. In this regard, it's hard to find a universal optimization for the whole multi-modal applications, thus they should be analyzed respectively to identify the bottleneck.

4.3 synchronization analysis

As Figure 5 shows, for all the applications we choose, the multi-modal implementations (multi) possess longer proportion of CPU+Runtime operation compared with the uni-modal implementations. While most of network computation are executed on GPU, we consider that a higher

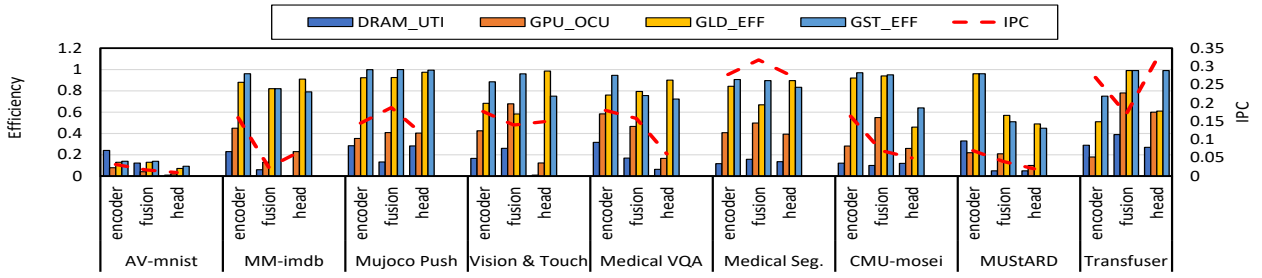


Figure 3: Resource usage of the three stages for different MMBench applications

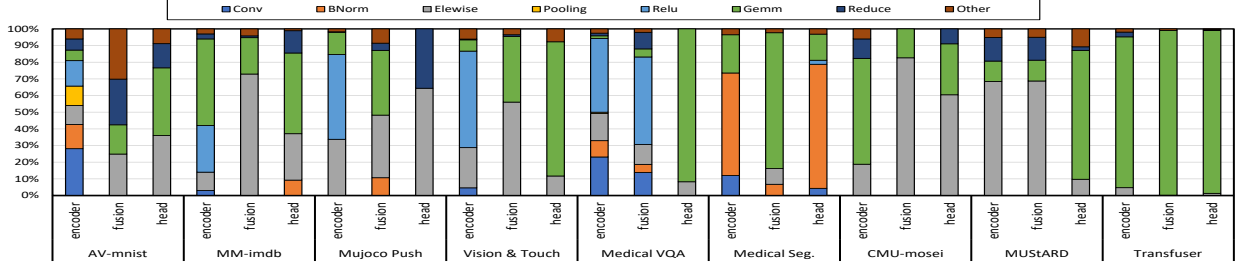


Figure 4: Kernel operation breakdown of three stages for different MMBench applications

CPU+Runtime ratio indicates more data synchronization and lower GPU utilization. So we can conclude that multi-modal DNNs suffer from data synchronization overhead. What's more, Figure 6 indicates that the execution time of different modalities(encoder) are different. So if all the encoders are executed in parallel, the faster encoder will have to wait for slower ones, causing resource under-utilization. This means the multi-modal DNNs also suffer from modality synchronization overhead.

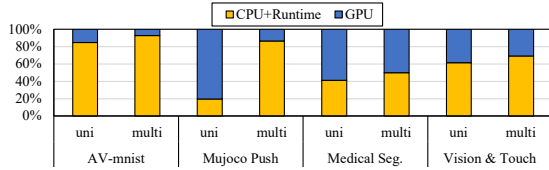


Figure 5: Comparison between synchronization and computation for MMBench applications

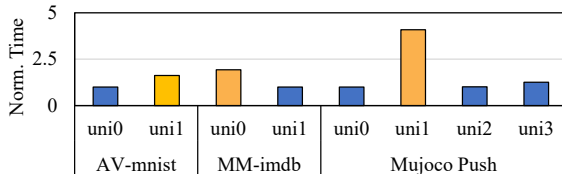


Figure 6: Execution time for different modalities for MMBench applications

5. CONCLUSION

After careful analysis taking advantage of these metrics, we find three remarkable characteristics:

Three-stage Execution. encoder, fusion and head stage perform very differently in the regard of GPU resource usage

because of the different operations or functions applied.

Frequent Synchronization. There exists extra synchronization in multi-modal DNNs. CPUs have to deal with data processing operations, such as *to* and *copy*. In the fusion stage, the fusion network must wait for the completion of all modalities, and additional CPU-GPU synchronization is needed to process intermediate data, such as the feature maps generated from various modalities.

High Heterogeneous. Since multi-modal networks apply suitable models to process various modalities in encoder stage and different fusion methods to federate the features of different modalities in fusion stage for different accuracy targets, the proportion of different kernel operations differs in both modalities and stages.

6. ACKNOWLEDGEMENTS

This work was done under the guidance of Cheng Xu and Professor Xiaofeng Hou.

7. REFERENCES

- [1] Arevalo, John, et al. "Gated multimodal units for information fusion". arXiv preprint arXiv:1702.01992 (2017).
- [2] Lew, Jonathan, et al. "Analyzing machine learning workloads using a detailed GPU simulator." ISPASS, 2019.
- [3] Parashar, Angshuman, et al. "Timeloop: A systematic approach to DNN accelerator evaluation." ISPASS, 2019.
- [4] Reddi, Vijay Janapa, et al. "MLPerf inference benchmark." ISCA, 2020.
- [5] Zhu, Hongyu, et al. "TBD: Benchmarking and analyzing deep neural network training." arXiv preprint arXiv:1803.06905 (2018).
- [6] Zhang, Qiyang, et al. "A comprehensive benchmark of deep learning libraries on mobile devices." Proceedings of the ACM Web Conference 2022.